



Universidad Nacional  
**Federico Villarreal**

# Sistema de Alerta Temprana de Fatiga y Somnolencia Multimodal para Choferes Basado

## Integrantes:

- Ruiz Meza Ricardo Antonio
- Vizcardo Zegarra Roberto Carlos
- Picon Ayala Yan Levi

## Docente

- Villafuerte Hernan

## Sección: “C”

**Año : 2026**



# Índice de Contenidos

**01** Arquitectura del  
Microcontrolador y Mapa de  
Pines

**02** Gestión de Memoria y Manejo  
de Búfers

**03** Diagrama de Flujo - Fase 1:  
Inicialización

**04** Diagrama de Flujo - Fases 2 y 3

**05** Diagrama de Flujo - Fase 4

**06** Código Fuente Parcial

# DESCRIPCIÓN DEL PROBLEMA

## **Incidencia Crítica**

Alta tasa de accidentes de tránsito en transporte interprovincial y de carga pesada debido al cansancio y horarios nocturnos.

## **Carencia Actual**

Los vehículos comerciales carecen de sistemas de monitoreo continuo en tiempo real dentro de la cabina que alerten de forma oportuna.

## **El Riesgo de Microsueños**

Periodos de escasos segundos donde el conductor pierde la conciencia por completo al volante sin darse cuenta.





# ARQUITECTURA DEL SISTEMA

## Unidad de Procesamiento Central

*Tarjeta ESP32-CAM basada en un procesador Tensilica Xtensa de 32 bits Dual-Core a 240 MHz.*

*ESP32-C3 SuperMini, chip de un solo nucleo*

## Módulo de Captura Óptica

*Sensor de video OV2640 acoplado con un diodo LED infrarrojo para visión nocturna en cabina.*

## Actuador Físico de Alerta

*Zumbador o Buzzer de 5V enlazado directamente al pin digital GPIO 2 del chip.*

## Estación de Trabajo

*Computadora encargada del análisis inteligente externo mediante el lenguaje Python.*

*Sensor BIOMEDICO MAX 30102*

*Modulo CARGADOR TP4056*

DIAGRAMA DE BLOQUES DEL SISTEMA DE ALERTA TEMPRANA DE FATIGA Y SOMNOLENCIA - FIIS UNFV

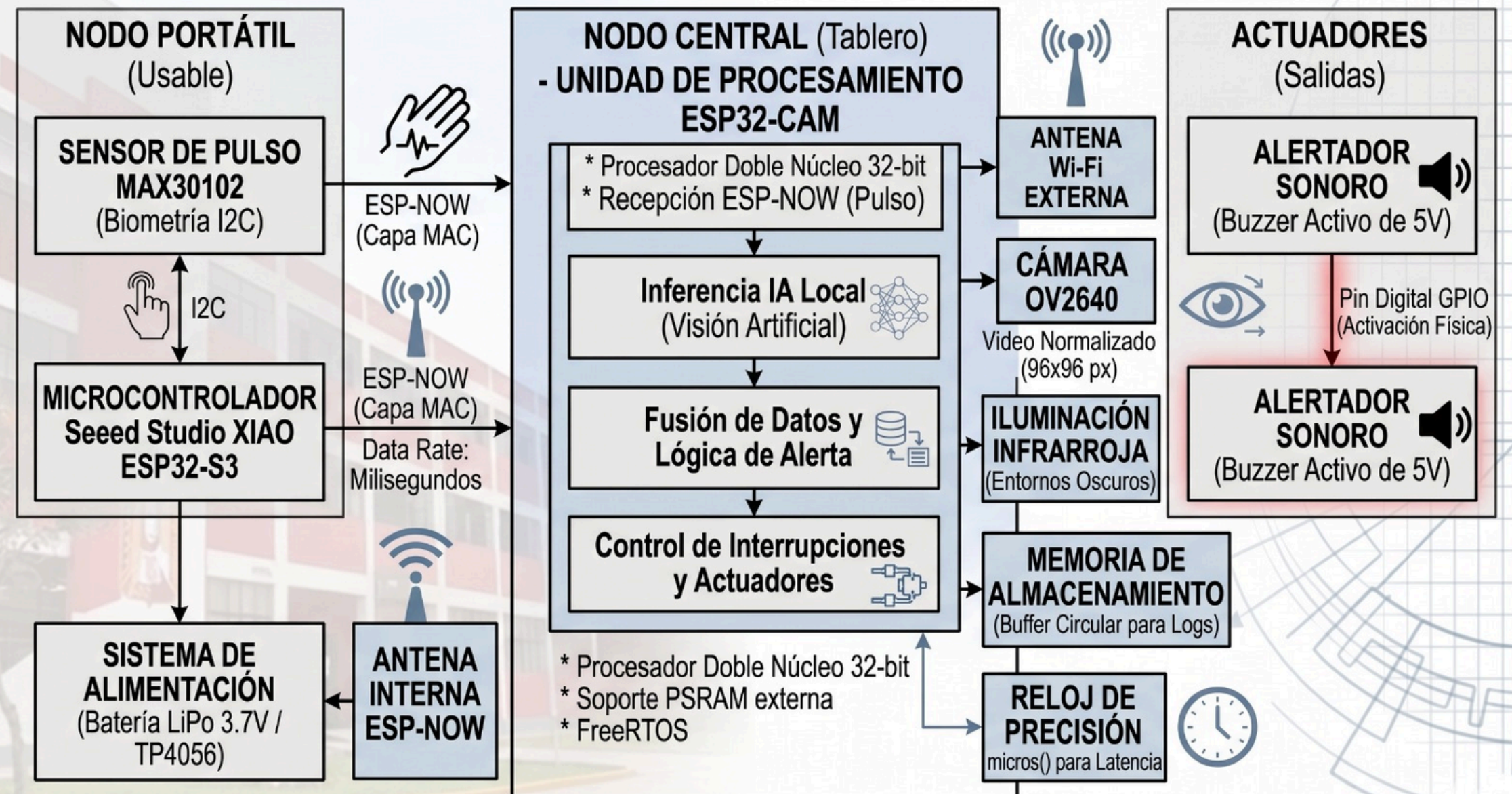


Fig. 1. Diagrama de bloques: Sensores, microcontrolador/CPU, actuadores, interfaces de comunicación.



# ARQUITECTURA DEL MICROCONTROLADOR Y MAPA DE PINES

## Procesador Central

*Chip Tensilica Xtensa de 32 bits Dual-Core a 240 MHz  
(ESP32-CAM)*

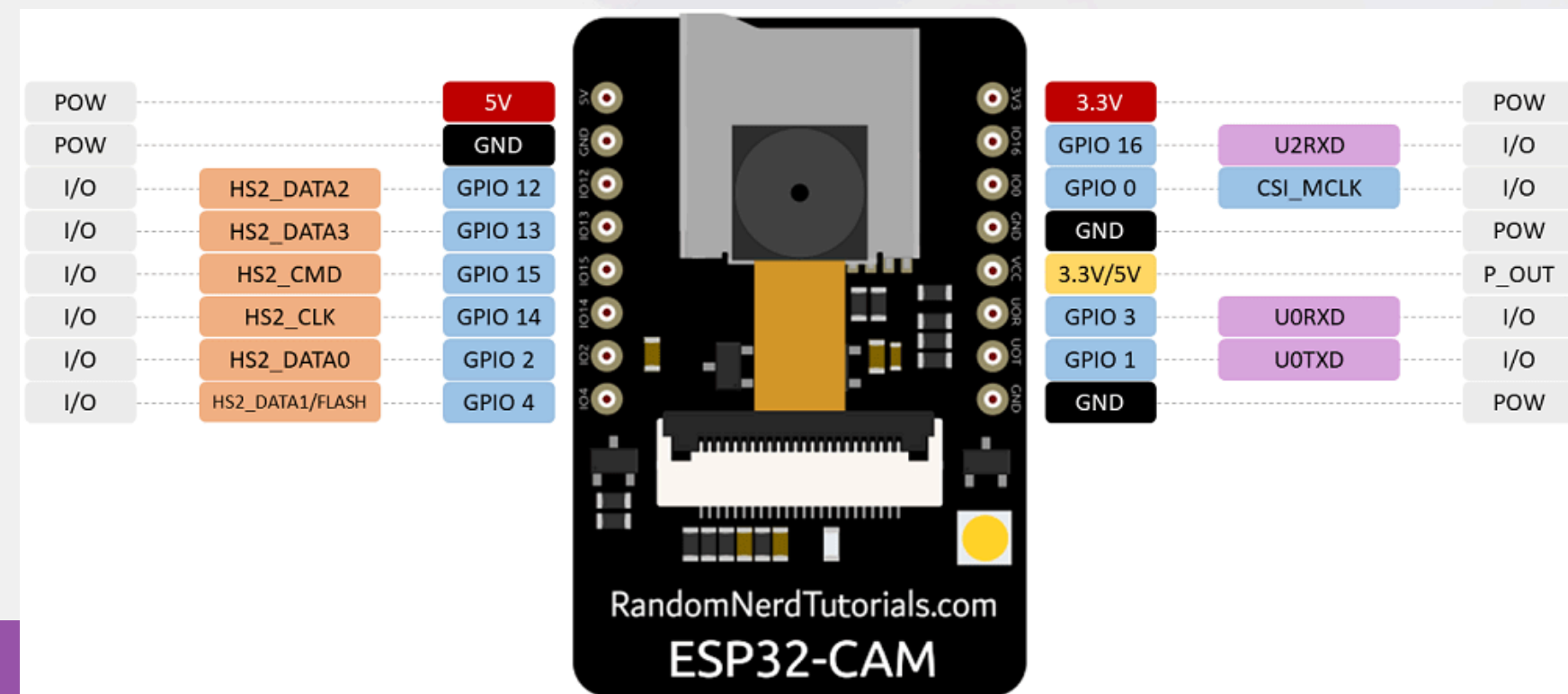
## Procesamiento Asíncrono

*Un núcleo se dedica exclusivamente a sostener la pila de red Wi-Fi y el streaming, mientras el segundo núcleo atiende las solicitudes de alerta del buzzer sin congelar el video.*

## Mapa de Pines (Hardware Base)

- *Captura Óptica: Sensor de video OV2640 y diodo LED Infrarrojo.*
- *Conexión de Programación: Pines RX/TX enlazados al adaptador serial FTDI de color rojo (interfaz UART a 115,200 baudios).*
- *Actuador Físico: Pin digital GPIO 2 configurado como salida para el buzzer de 5V.*

```
4
5 // =====
6 // 1. PINES (AI-THINKER) - MAPEO DE HARDWARE BASE
7 // =====
8
9 #define PWDN_GPIO_NUM    32
10 #define RESET_GPIO_NUM  -1
11 #define XCLK_GPIO_NUM    0
12 #define SIOD_GPIO_NUM    26
13 #define SIOC_GPIO_NUM    27
14 #define Y9_GPIO_NUM      35
15 #define Y8_GPIO_NUM      34
16 #define Y7_GPIO_NUM      39
17 #define Y6_GPIO_NUM      36
18 #define Y5_GPIO_NUM      21
19 #define Y4_GPIO_NUM      19
20 #define Y3_GPIO_NUM      18
21 #define Y2_GPIO_NUM       5
22 #define VSYNC_GPIO_NUM   25
23 #define HREF_GPIO_NUM    23
24 #define PCLK_GPIO_NUM    22
25 #define BUZZER_PIN        2 // Pin GPIO 2 enlazado al buzzer de 5V
26
27 const char* ssid      = "MOVISTAR_CF60";
28 const char* password = "Fy5F7BFKxNAfdghzNTvt";
29
```



# GESTIÓN DE MEMORIA Y MANEJO DE BÚFERS

## Limitación de Hardware

*Los 520 KB de la memoria SRAM interna son insuficientes para procesar video y causarían un colapso del sistema (Crash)*

## Estrategia de Diseño (Búfer Masivo)

*Un núcleo se dedica exclusivamente a sostener la pila de red Wi-Fi y el streaming, mientras el segundo núcleo atiende las solicitudes de alerta del buzzer sin congelar el video.*

## Flujo de Datos a Nivel de Búfer

*La función nativa esp\_camera\_fb\_get() captura la foto desde el lente OV2640 en resolución ligera QVGA (320x240) y aloja los bytes en formato JPEG temporalmente en la PSRAM.*

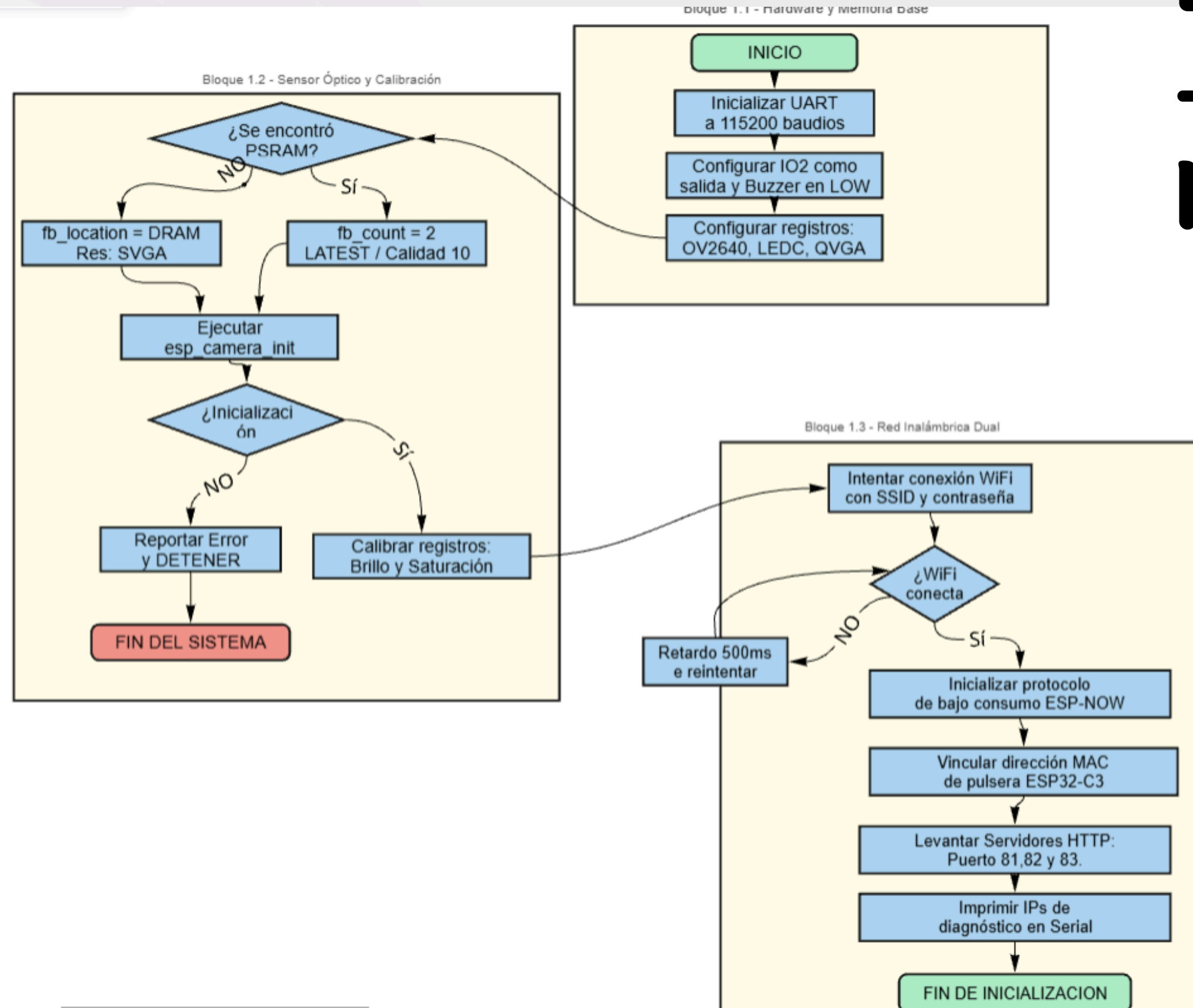
## Protección del Sistema

*Al liberar la memoria interna, la SRAM queda disponible únicamente para las variables globales, contraseñas de red y las pilas de trabajo del servidor web.*

| Componente de Memoria | Capacidad Física | Función Principal en el Proyecto                                                                         |
|-----------------------|------------------|----------------------------------------------------------------------------------------------------------|
| Memoria SRAM Interna  | 520 KB           | Guarda variables globales, contraseñas de red y controla los hilos del servidor web.                     |
| Memoria PSRAM Externa | 4 MB             | Actúa como el buffer masivo donde se guardan temporalmente las fotos de la cámara antes de transmitirse. |

# DIAGRAMA DE FLUJO

## - FASE 1: INICIALIZACIÓN



*Calibración Óptica:* Se arranca el sensor OV2640. El chip ajusta de forma automática los registros de brillo y contraste para garantizar una visión nítida en la cabina nocturna mediante el LED infrarrojo.

*Gestión de Memoria y Diagnóstico:* La tarjeta inicia la comunicación serial a 115,200 baudios. Simultáneamente, reserva el búfer de video en la memoria PSRAM externa de 4 MB, protegiendo la SRAM interna contra desbordamientos.

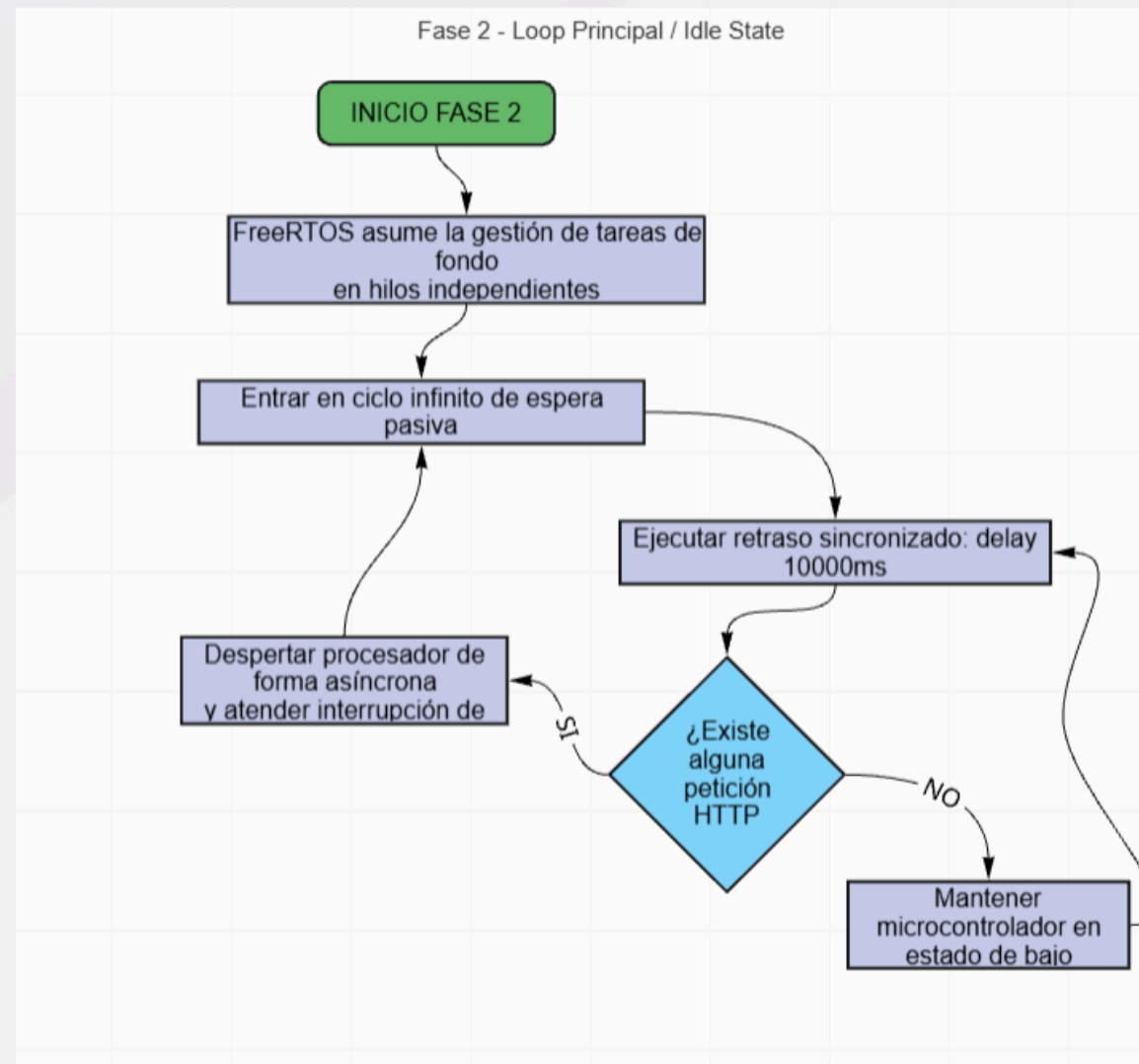
*Despliegue de Red Dual (Actualizado):* La tarjeta activa la red Wi-Fi y el protocolo de bajo consumo ESP-NOW (para enlazarse con la pulsera). Inmediatamente, levanta tres servidores HTTP asíncronos: Puerto 81 (Video), Puerto 82 (Alerta acústica) y Puerto 83 (Telemetría cardíaca).



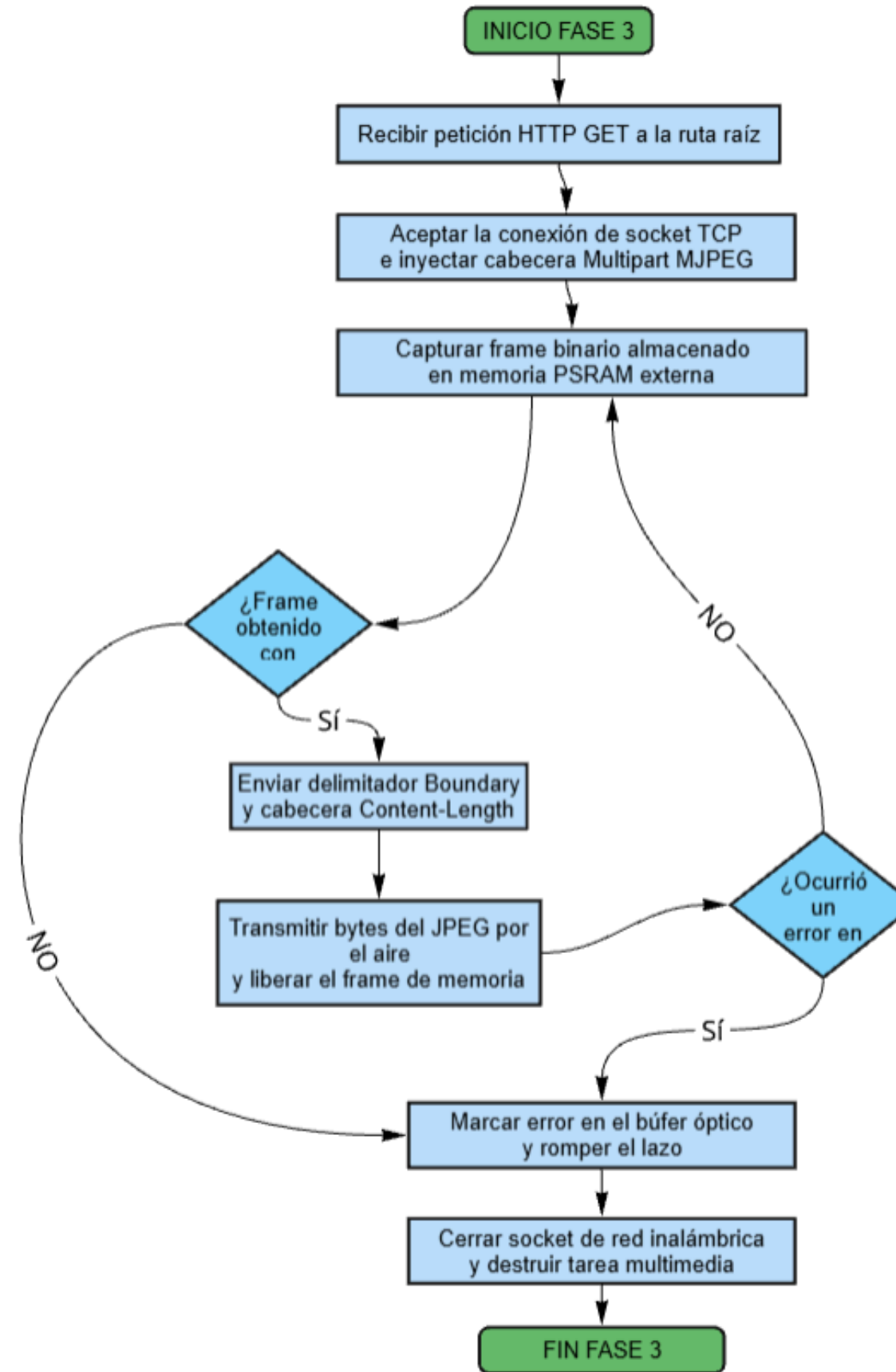
# DIAGRAMA DE FLUJO - FASES 2 Y 3

## Fase 2

**Gestión Asíncrona (FreeRTOS):** El void loop() se mantiene en estado de bajo consumo (delay(10000ms)). El kernel del sistema operativo FreeRTOS asume el control en paralelo de los tres puertos sin saturar la CPU.



Fase 3 - Servidor de Streaming de Video - Puerto 81



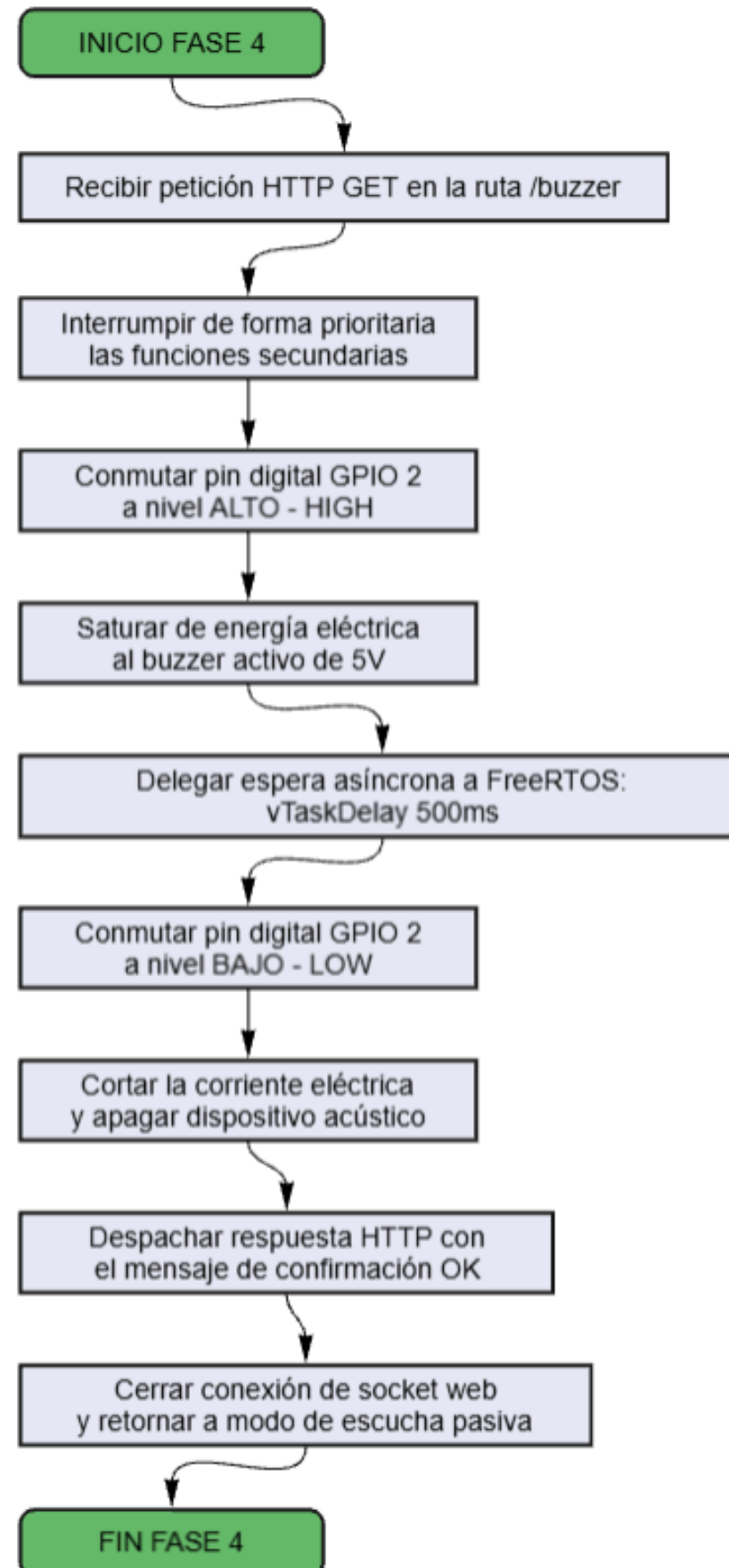
**Streaming de Video (Puerto 81):** Al recibir la petición HTTP GET desde Python, inicia la transmisión continua de imágenes bajo el protocolo Multipart MJPEG.

**Control de Búfer:** En cada ciclo, `esp_camera_fb_get()` captura el fotograma desde la PSRAM externa. Tras enviarlo por la red, se ejecuta `esp_camera_fb_return()` para vaciar la memoria al instante y evitar colapsos.

**Protección del Socket:** Ante cualquier desconexión TCP o error de lectura, el lazo se rompe drásticamente para proteger los recursos del microcontrolador.

# DIAGRAMA DE FLUJO – FASE 4

Fase 4 - Servidor de Control de Alarma - Puerto 82



- **Recepción Biométrica (Puerto 83):** De forma silenciosa y pasiva, este puerto despacha a Python los datos de frecuencia cardíaca (BPM) que llegan desde la pulsera inteligente.

- **Retención de Seguridad: FreeRTOS** sostiene el sonido por exactamente 500 milisegundos mediante `vTaskDelay()`. Esto garantiza que el chofer despierte sin congelar la transmisión del video, retornando luego a la escucha pasiva.

**Fusión de Variables (Actualizado):** Si el Cerebro Analítico en Python detecta una coincidencia total (ojos cerrados por > 2 segundos Y una caída del pulso basal > 15%), un hilo secundario dispara un HTTP GET a la ruta /buzzer del Puerto 82.

- **Interrupción de Hardware:** Al recibir la orden de emergencia, la rutina conmuta el pin digital GPIO 2 a nivel ALTO, encendiendo físicamente el buzzer de 5V de forma inmediata.

# PROCESAMIENTO DE VIDEO Y LÓGICA DE ALERTA (PYTHON)

## Optimización del Flujo Visual

*Captura del rostro del conductor en resolución ligera QVGA. Se aplica un filtro lógico que descarta una de cada dos fotos para reducir drásticamente el esfuerzo de la CPU.*

## Detección de Microsueños

*El estado de emergencia se activa de forma automática si el algoritmo detecta que los ojos permanecen cerrados (EAR por debajo del umbral crítico de 0.22) por más de 2 segundos seguidos.*

## Arquitectura Multihilo

*Al detectar la somnolencia, Python crea un hilo de trabajo separado exclusivo para enviar la petición rápida HTTP GET (/buzzer) al puerto 82 de la tarjeta.*

## Malla Facial y Métrica EAR

*Integración de la herramienta MediaPipe de Google para identificar puntos de referencia faciales. Uso de la fórmula matemática Eye Aspect Ratio (EAR) para calcular en tiempo real la apertura promedio de los párpados.*

```
33 # 3. FUNCIÓN DE CÁLCULO MATEMÁTICO DEL EAR
34 # =====
35 def calcular_ear(puntos_ojo, landmarks, ancho, alto):
36     # Traduce las coordenadas relativas flotantes a píxeles absolutos en la RAM
37     coords = [
38         np.array([landmarks[p].x * ancho, landmarks[p].y * alto])
39         for p in puntos_ojo
40     ]
41     # Distancias Euclidianas verticales y horizontales de la geometría ocular
42     d_vertical1 = np.linalg.norm(coords[1] - coords[5])
43     d_vertical2 = np.linalg.norm(coords[2] - coords[4])
44     d_horizontal = np.linalg.norm(coords[0] - coords[3])
45     # Fórmula matemática de la Relación de Aspecto del Ojo
46     return (d_vertical1 + d_vertical2) / (2.0 * d_horizontal)
47
```

## Beneficio Técnico Central

*La ejecución en paralelo mediante hilos garantiza que el envío de la orden de alerta se realice de inmediato, sin retrasar ni congelar la transmisión de la cámara.*



# GESTIÓN AVANZADA DE MEMORIA

## Memoria SRAM Interna (520 KB)

*Recursos limitados protegidos y destinados solo a variables globales, contraseñas de red y control de sockets.*

## Uso Estratégico de PSRAM Externa (4 MB)

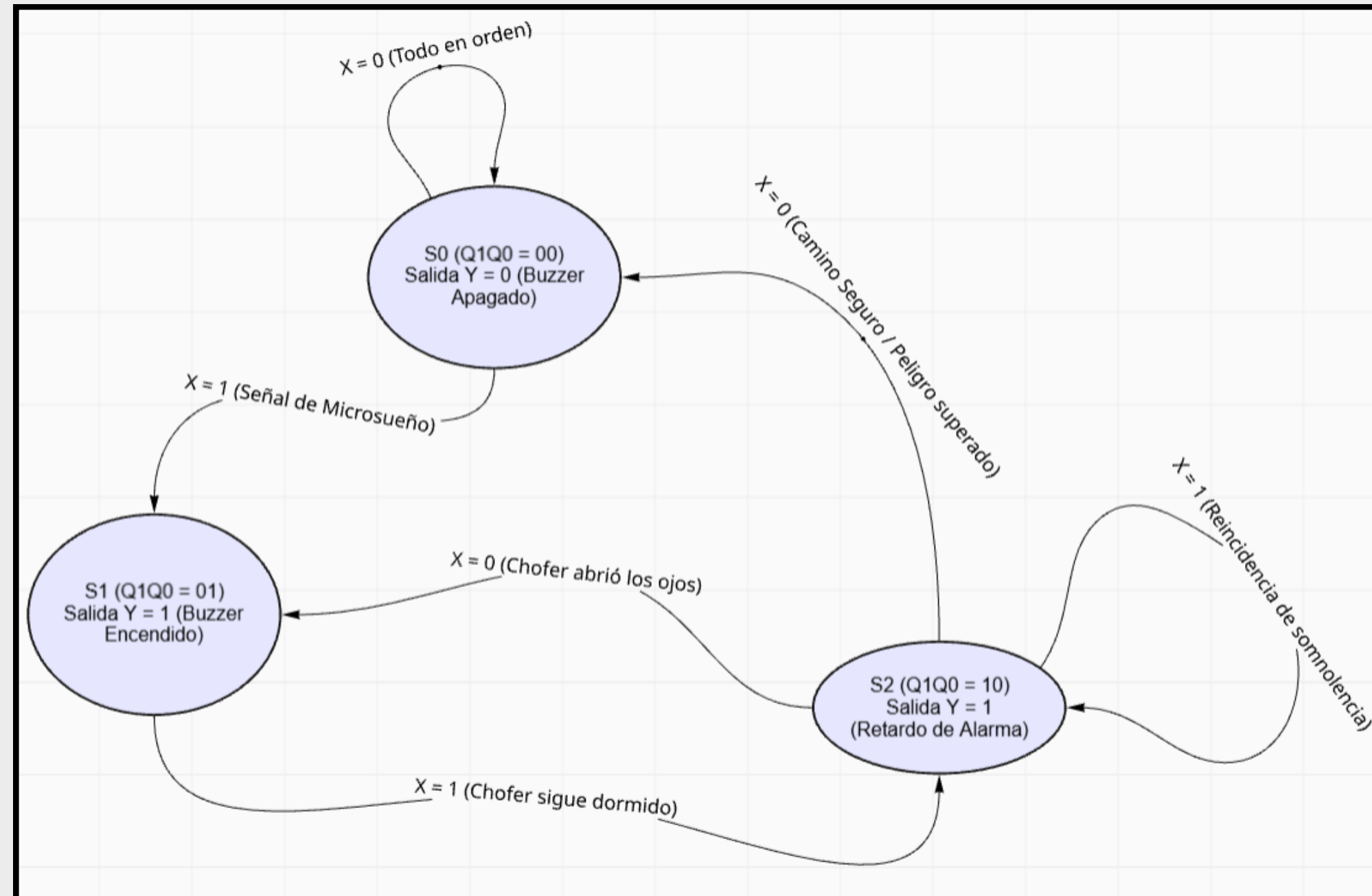
*Mapeo obligatorio del buffer masivo donde la función nativa `esp_camera_fb_get()` guarda temporalmente las fotos JPEG.*

## Mitigación de Riesgos

*El uso de memoria externa alivia la SRAM interna evitando colapsos del sistema (Crash) por falta de espacio durante el streaming.*

| Componente de Memoria | Capacidad Física | Función Principal en el Proyecto                                                                         |
|-----------------------|------------------|----------------------------------------------------------------------------------------------------------|
| Memoria SRAM Interna  | 520 KB           | Guarda variables globales, contraseñas de red y controla los hilos del servidor web.                     |
| Memoria PSRAM Externa | 4 MB             | Actúa como el buffer masivo donde se guardan temporalmente las fotos de la cámara antes de transmitirse. |

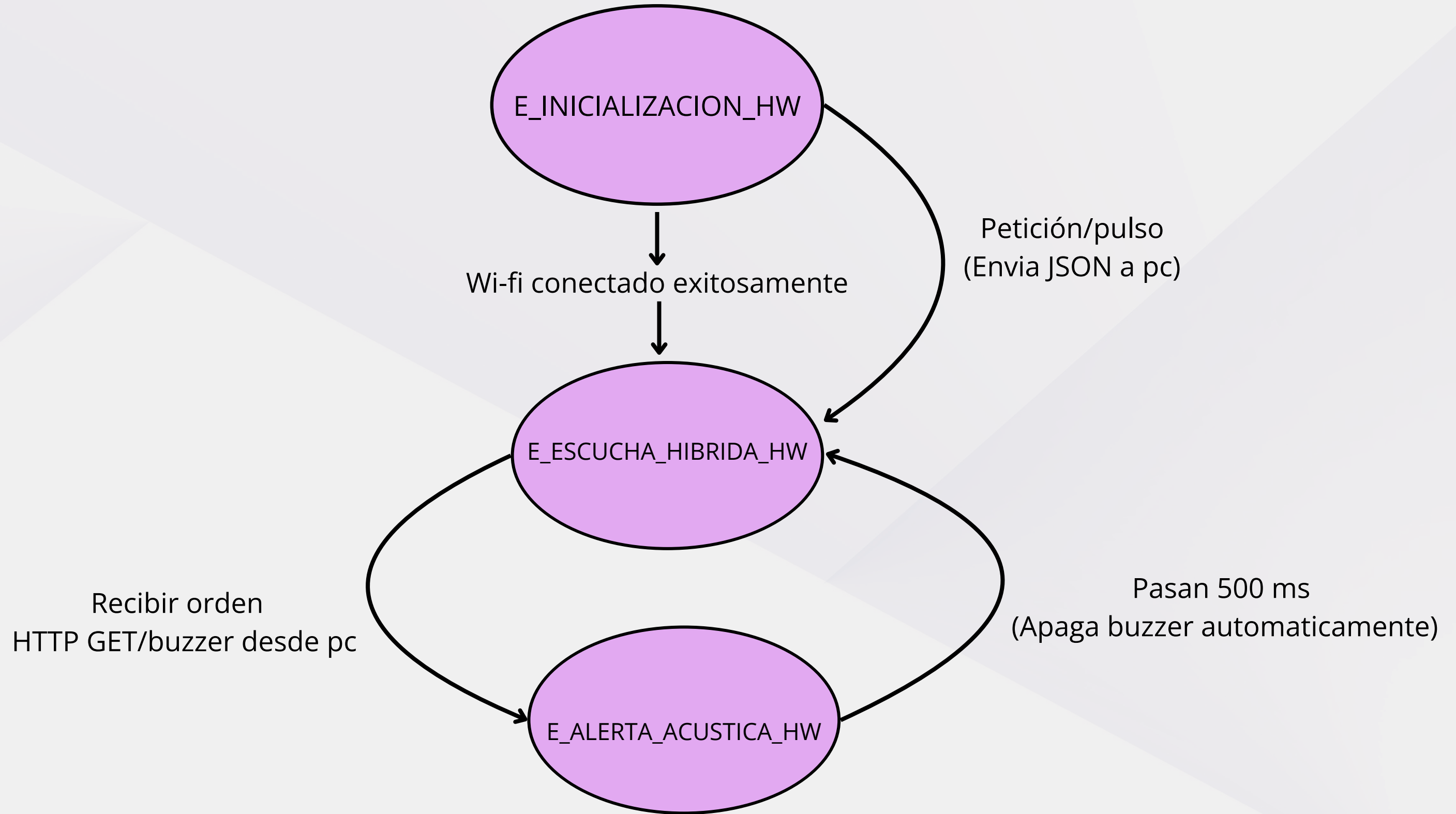
# LÓGICA DE CONTROL DE HARDWARE (ACTUADOR Y TABLERO)



**Diseño Secuencial:** El comportamiento del zumbador está regido por una máquina de estados finitos que evita ciclos infinitos o bloqueos.

- **Estado S0 (Reposo Nominal):** El conductor está alerta ( $X=0$ ). La salida se mantiene baja ( $Y=0$ , Buzzer apagado).
- **Estado S1 (Activación Inmediata):** Al recibir la señal de microsueño por red ( $X=1$ ), transita en milisegundos a nivel alto ( $Y=1$ , Buzzer encendido).
- **Estado S2 (Retención y Seguridad):** Retardo asíncrono (500ms gestionados por FreeRTOS). Garantiza que la alarma suene lo suficiente para despertar al chofer antes de evaluar si el peligro ya fue superado ( $X=0$ ).

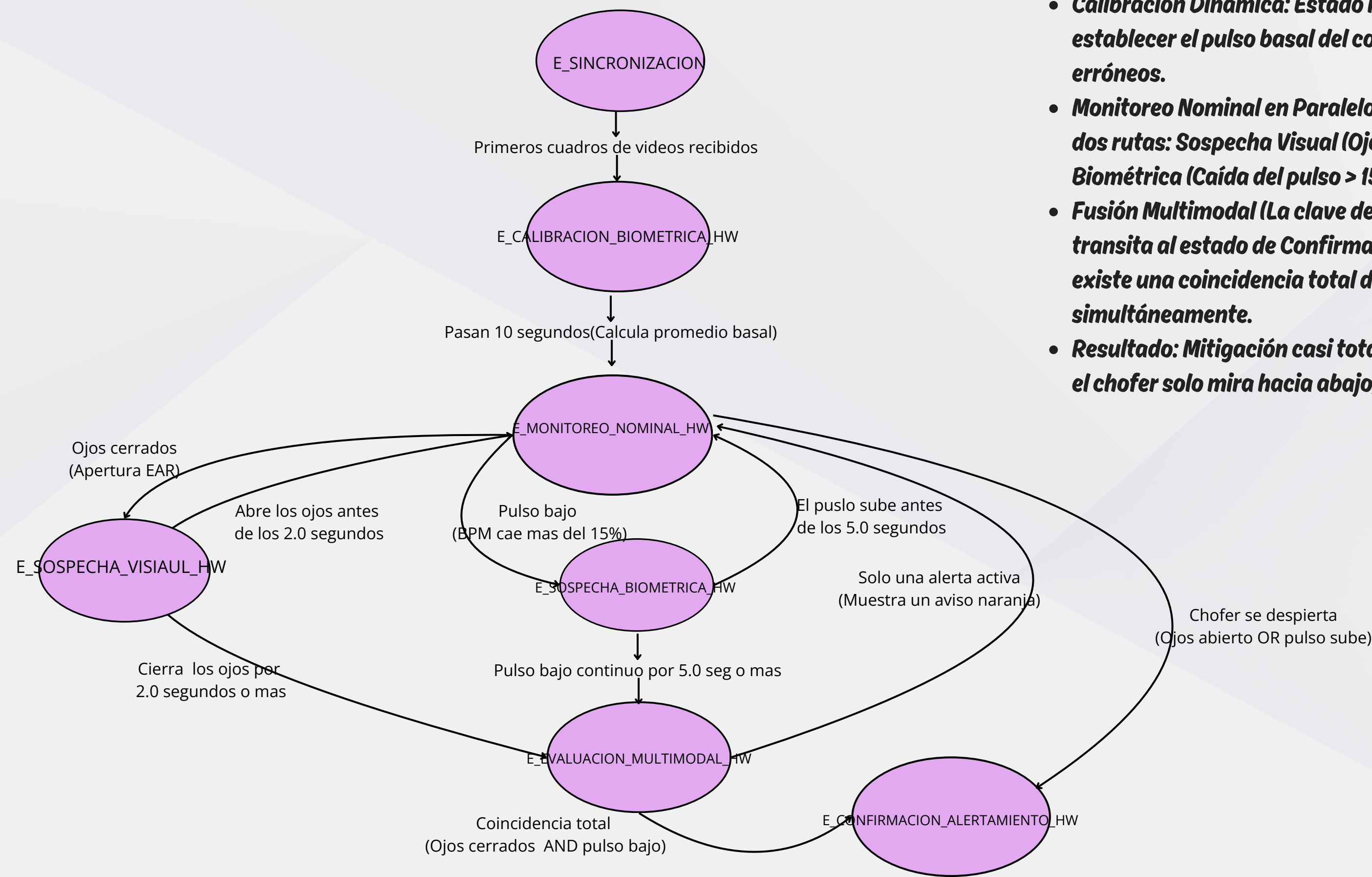
# MAQUINA DE TABLERO





# MAQUINA CEREBRO ANALITICO

- **Calibración Dinámica:** Estado inicial de 10 segundos para establecer el pulso basal del conductor y evitar umbrales fijos erróneos.
- **Monitoreo Nominal en Paralelo:** El sistema evalúa continuamente dos rutas: **Sospecha Visual (Ojos cerrados > 2 seg)** y **Sospecha Biométrica (Caída del pulso > 15%)**.
- **Fusión Multimodal (La clave del sistema):** El algoritmo solo transita al estado de **Confirmación de Alerta** y dispara la red si existe una **coincidencia total de ambas variables simultáneamente**.
- **Resultado:** Mitigación casi total de falsas alarmas (por ejemplo, si el chofer solo mira hacia abajo, el estado retorna a Reposo).



# LISTA DE MÉTRICAS TÉCNICAS

## Tiempo de Reacción de la Alarma

*Medición en milisegundos (ms) desde el envío de la orden web en Python hasta la activación eléctrica del buzzer físico.*

## LATENCIA DE MICROSUÑO

## Estabilidad de la Medición Ocular

*Validación de la precisión del reconocimiento del umbral EAR (0.22) frente a perturbaciones de luz gracias al LED infrarrojo*

## Velocidad de Transmisión de Video

*Evaluación del rendimiento de la red inalámbrica local calculando los cuadros por segundo reales (FPS) procesados*

# VALIDACIÓN DE MÉTRICAS PRELIMINARES (HARDWARE Y RED)

## Precisión del Sensor Biomédico (MAX30102 vs. Oxímetro)

Objetivo: Demostrar que el prototipo lee el pulso real del conductor con un error menor al 5%.



| Muestra de Prueba | Oxímetro Clínico (BPM) | Pulsera Prototipo (BPM) | Diferencia (Error) |
|-------------------|------------------------|-------------------------|--------------------|
| Lectura 1         | 84                     | 82                      | 2 BPM              |
| Lectura 2         | 86                     | 83                      | 3 BPM              |
| Lectura 3         | 81                     | 81                      | 0 BPM              |
| Lectura 4         | 88                     | 87                      | 1 BPM              |
| Lectura 5         | 85                     | 85                      | 0 BPM              |
| Promedio Final    | 84.8 BPM               | 83.6 BPM                | < 1.41 %           |

## Latencia de detección de microsueño

| Parámetro                                       | Valor esperado  | Valor real del sistema                           |
|-------------------------------------------------|-----------------|--------------------------------------------------|
| Tiempo desde cierre de ojos hasta alerta visual | <=2 segundos    | 2.0 segundos (configurado en TIEMPO_ALERTA_OJOS) |
| Tiempo adicional hasta activación del buzzer    | <= 0.5 segundos | ~0.5 segundos (vTaskDelay(500ms) en ESP32-CAM)   |
| Latencia total extremo a extremo                | <= 3 segundos   | ~2.5 segundos                                    |



# CONSUMO Y OPTIMIZACIÓN DE RECURSOS

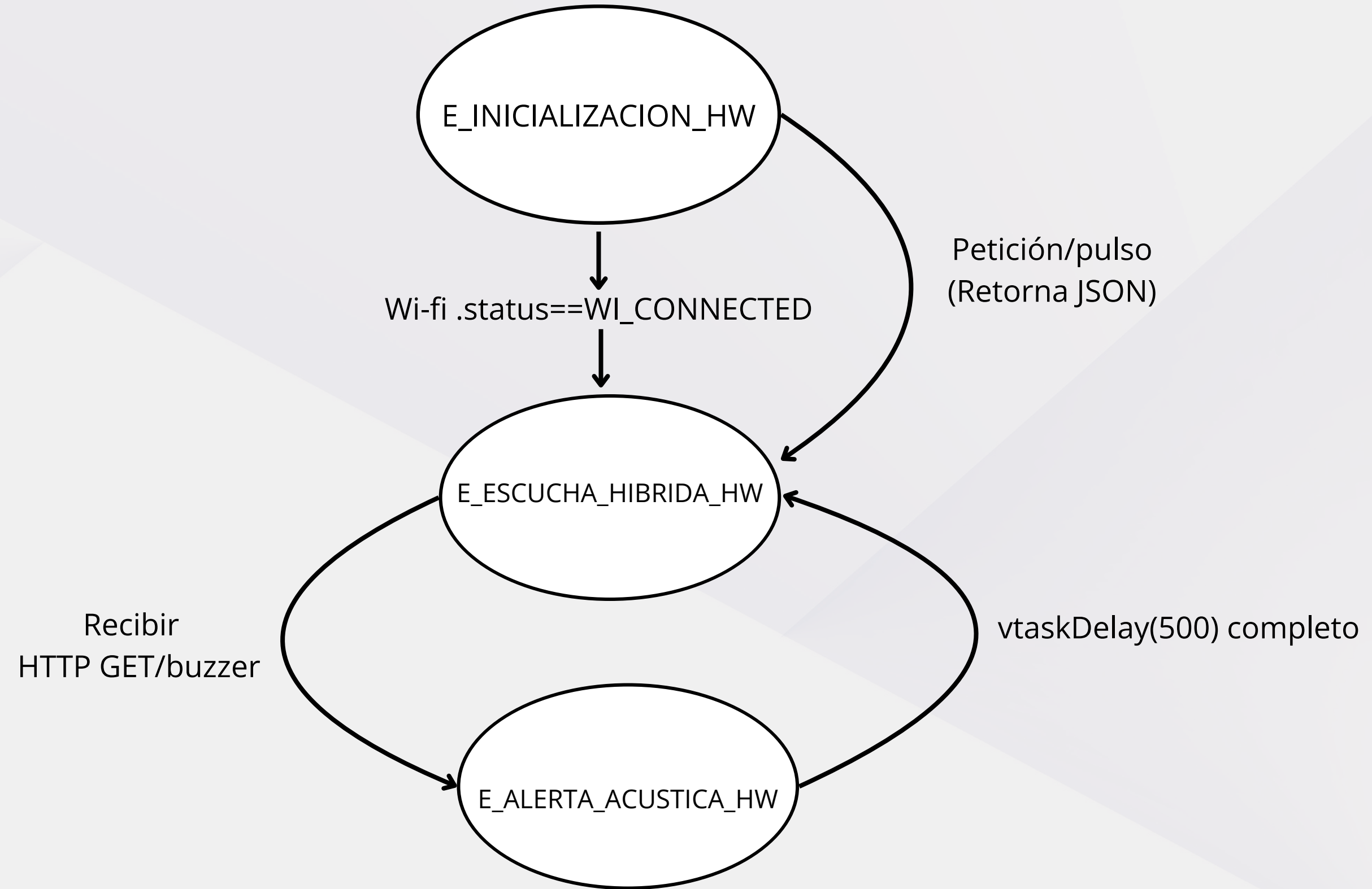
## Análisis de Memoria del Nodo Central (ESP32-CAM)

| Componente de Memoria   | Capacidad Total | Uso Medido (KB) | Porcentaje (%) | Diagnóstico / Estado                               |
|-------------------------|-----------------|-----------------|----------------|----------------------------------------------------|
| Memoria Flash (Código)  | 3 MB            | 820 KB          | 26%            | Firmware optimizado. Espacio libre para futura IA. |
| Memoria SRAM (Interna)  | 520 KB          | 47 KB           | 14%            | Sin riesgo de desbordamiento (Stack Overflow).     |
| Memoria PSRAM (Externa) | 4 MB            | 4 MB (Activa)   | 100%           | Soporta con éxito el búfer de imágenes pesadas.    |

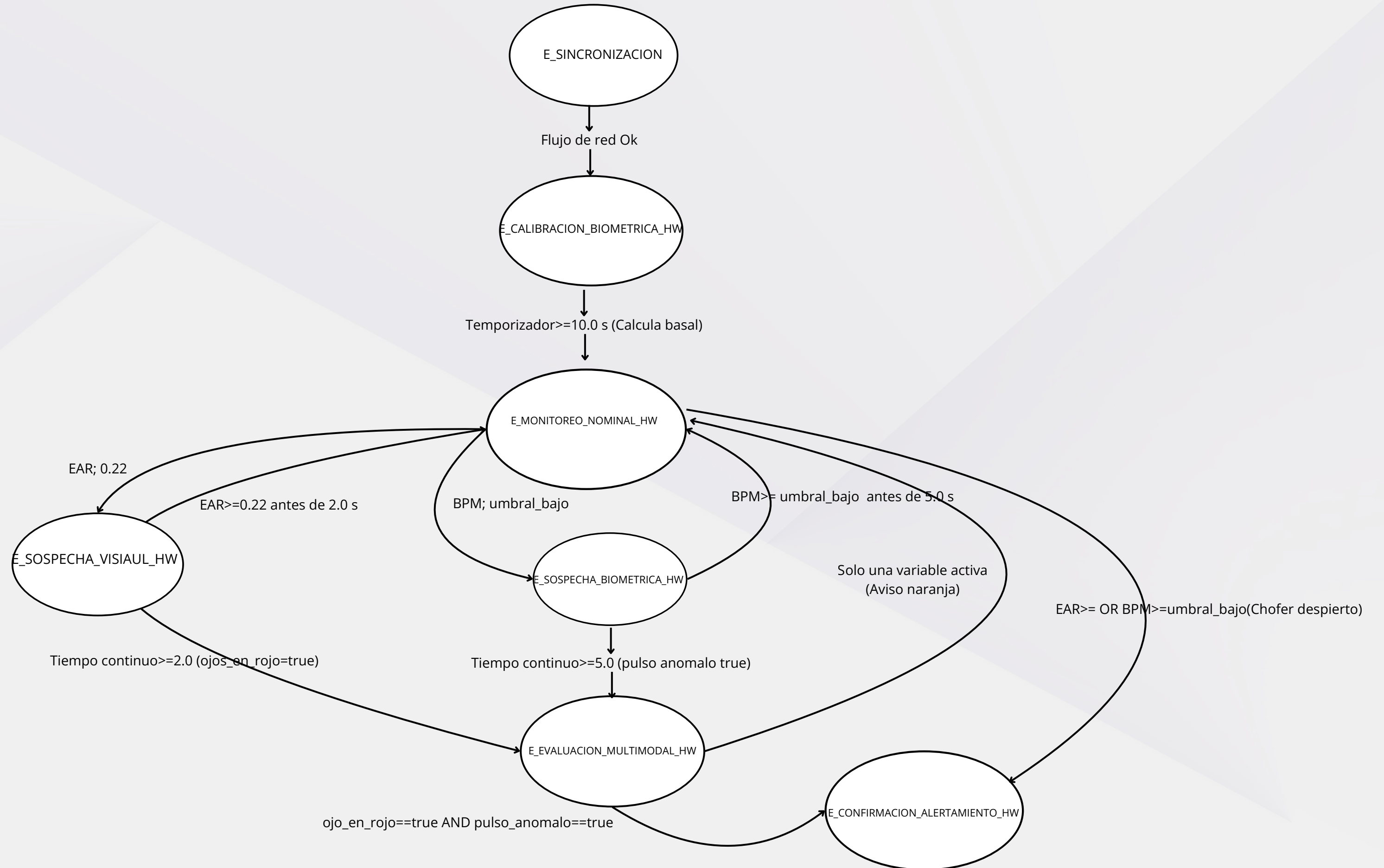
El Sketch usa 839421 bytes (26%) del espacio de almacenamiento de programa. El máximo es 3145728 bytes.  
Las variables Globales usan 48072 bytes (14%) de la memoria dinámica, dejando 279608 bytes para las variables locales. El máximo es 327680 bytes.

El Sketch usa 719812 bytes (54%) del espacio de almacenamiento de programa. El máximo es 1310720 bytes.  
Las variables Globales usan 37860 bytes (11%) de la memoria dinámica, dejando 289820 bytes para las variables locales. El máximo es 327680 bytes.

# TABLERO (ESP32-CAM)

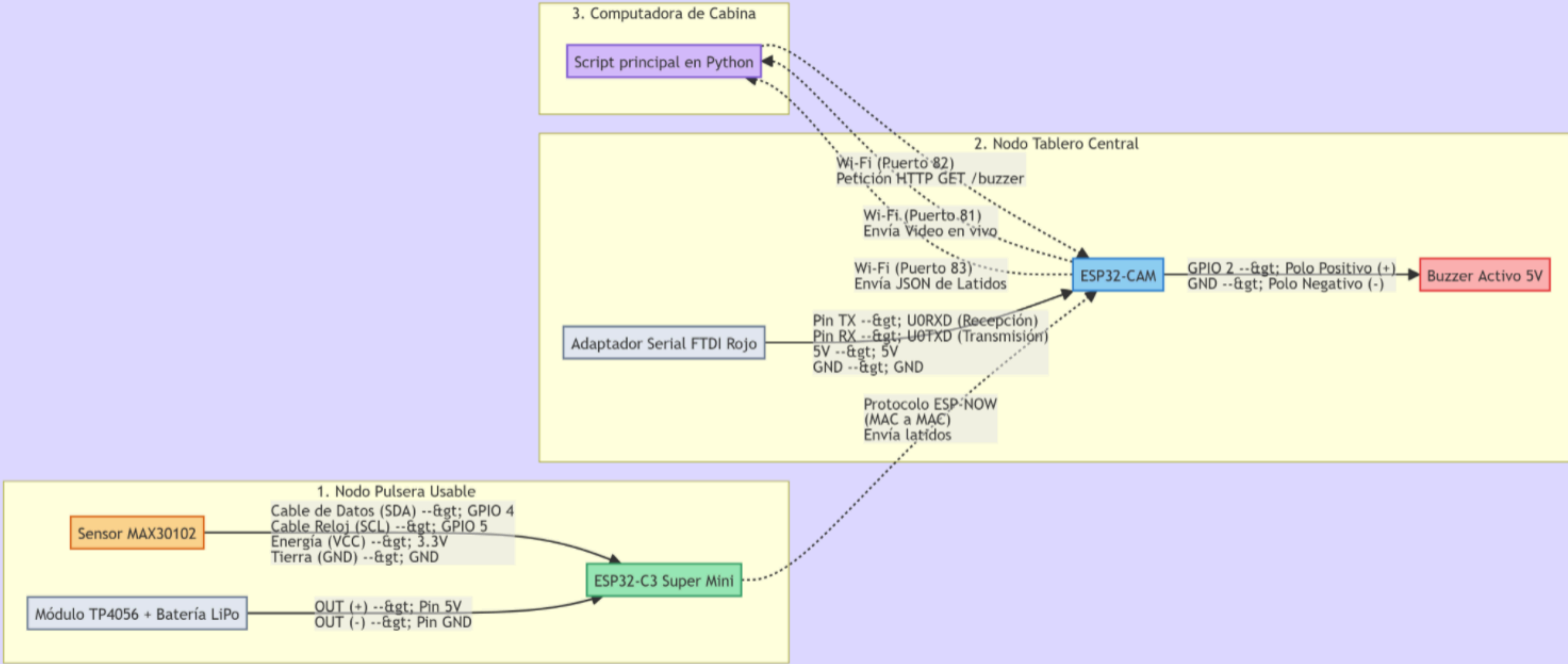


# CEREBRO ANALITICO(PC PYTHON)





# Conexiones de componentes



# ¿QUÉ ES SOMNISENSE?

SomniSense es un sistema multimodal de alerta temprana que combina visión artificial, telemetría inercial y un aplicativo móvil Android para detectar fatiga en tiempo real y activar una alarma física antes de que el conductor pierda el control del vehículo

## TRES PILARES:

### **VISIÓN ARTIFICIAL**

*Analiza el rostro del conductor frame a frame usando MediaPipe FaceLandmarker con 478 landmarks faciales. Calcula la apertura ocular (EAR) y la apertura bucal (MAR) en cada fotograma del video recibido del tablero.*

### **TELEMETRÍA INERCIAL**

*El sensor MPU6050 en la gorra mide el ángulo de cabeceo (pitch) a 50 lecturas por segundo con un filtro complementario de alta precisión. Detecta la atonía muscular cervical característica del microsueño.*

### **APP ANDROID**

*Es el único componente con inteligencia del sistema. Recibe las tres fuentes de datos, aplica las cinco métricas y decide si corresponde activar la alarma física en la gorra. Todo ocurre en tiempo real y sin conexión a internet.*



# ¿QUÉ ES SOMNISENSE?

SomniSense es un sistema multimodal de alerta temprana que combina visión artificial, telemetría inercial y un aplicativo móvil Android para detectar fatiga en tiempo real y activar una alarma física antes de que el conductor pierda el control del vehículo

## TRES PILARES:

### **VISIÓN ARTIFICIAL**

*Analiza el rostro del conductor frame a frame usando MediaPipe FaceLandmarker con 478 landmarks faciales. Calcula la apertura ocular (EAR) y la apertura bucal (MAR) en cada fotograma del video recibido del tablero.*

### **TELEMETRÍA INERCIAL**

*El sensor MPU6050 en la gorra mide el ángulo de cabeceo (pitch) a 50 lecturas por segundo con un filtro complementario de alta precisión. Detecta la atonía muscular cervical característica del microsueño.*

### **APP ANDROID**

*Es el único componente con inteligencia del sistema. Recibe las tres fuentes de datos, aplica las cinco métricas y decide si corresponde activar la alarma física en la gorra. Todo ocurre en tiempo real y sin conexión a internet.*



# GESTIÓN AVANZADA DE MEMORIA

## Memoria SRAM Interna (520 KB)

*Recursos limitados protegidos y destinados solo a variables globales, contraseñas de red y control de sockets.*

## Uso Estratégico de PSRAM Externa (4 MB)

*Mapeo obligatorio del buffer masivo donde la función nativa `esp_camera_fb_get()` guarda temporalmente las fotos JPEG.*

## Mitigación de Riesgos

*El uso de memoria externa alivia la SRAM interna evitando colapsos del sistema (Crash) por falta de espacio durante el streaming.*

| Componente de Memoria | Capacidad Física | Función Principal en el Proyecto                                                                         |
|-----------------------|------------------|----------------------------------------------------------------------------------------------------------|
| Memoria SRAM Interna  | 520 KB           | Guarda variables globales, contraseñas de red y controla los hilos del servidor web.                     |
| Memoria PSRAM Externa | 4 MB             | Actúa como el buffer masivo donde se guardan temporalmente las fotos de la cámara antes de transmitirse. |





**Borcelle**

# **MUCHAS GRACIAS**

[www.unsitiogenial.es](http://www.unsitiogenial.es)